

Agent Factories for High Level Synthesis: How Far Can General-Purpose Coding Agents Go in Hardware Optimization?

Abhishek Bhandwalder, Mihir Choudhury, Ruchir Puri, Akash Srivastava
IBM Corporation

Abstract—We present an empirical study of how far general-purpose coding agents—with no hardware-specific training—can go in optimizing hardware designs from high-level algorithmic specifications. Our method, an *agent factory*, is a two-stage pipeline that builds and coordinates multiple autonomous optimization agents. In Stage 1, the factory decomposes a design into sub-kernels, independently optimizes each through pragma and code-level transformations, and solves an Integer Linear Program to assemble globally promising configurations under area constraints. In Stage 2, it spawns N expert agents over the top ILP solutions, each exploring cross-function optimizations—pragma recombination, loop fusion, memory restructuring—that sub-kernel decomposition cannot reach.

We evaluate the pipeline on twelve kernels from HLS-Eval and Rodinia-HLS using Claude Code (Opus 4.5/4.6) with AMD Vitis HLS. Scaling from 1 to 10 agents yields a mean $7.07\times$ speedup over baseline, with gains concentrated on harder problems: *leukocyte* and *kmeans* reaches $\sim 10\times$. Across benchmarks, agents consistently recover known hardware optimization patterns without domain-specific training, and winning designs frequently originate from non-top-ranked ILP variants, confirming that global optimization uncovers improvements invisible to sub-kernel search. These findings establish agent scaling as a practical axis for HLS optimization. We provide an anonymized implementation here.

Index Terms—Large Language Models (LLMs), Multi-Agent Systems, Agentic AI, Inference-Time Scaling, High-Level Synthesis (HLS), Design Space Exploration (DSE), Hardware Optimization

I. INTRODUCTION

High-Level Synthesis (HLS) strives to raise the abstraction of hardware design from RTL to C/C++, with the goal of achieving high performance. However, with current state-of-the-art HLS tools, significant expert-driven pragma insertion and code restructuring remains necessary to achieve desirable results [1]–[3]. Selecting the right combination of directives such as PIPELINE, UNROLL, ARRAY_PARTITION, and others requires deep hardware knowledge, careful latency-area tradeoff reasoning, and extensive iteration with the synthesis toolchain [4]. As an example, Cong et al. report that more than 40% of lines of high-level code in a real-world genomics kernel were attributable to hardware-specific optimizations and pragmas alone [4], illustrating the manual effort that persists even with modern HLS tools.

Existing automation approaches treat **high-level code to RTL generation** as a black-box optimization problem over a predefined parameter space. Bayesian optimization meth-

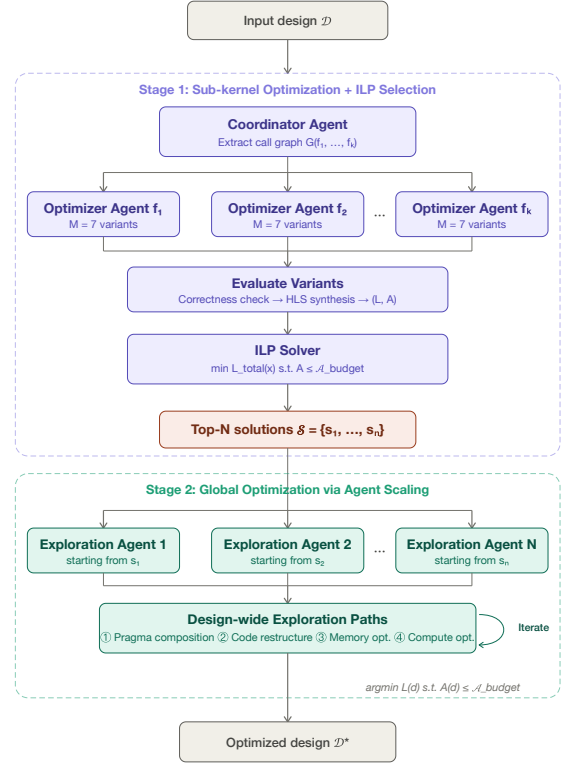


Fig. 1. Two-stage agent-based pipeline for HLS design space exploration. Given an input design $\mathcal{D}$, a coordinator agent extracts the function call graph G and spawns one optimizer agent per sub-function $f_1, \dots, f_K$. Variants are evaluated for correctness and synthesized to obtain (latency, area) pairs. An ILP solver then selects the top- N combinations $\mathcal{S} = \{s_1, \dots, s_N\}$ that minimize total latency subject to the area budget. In Stage 2, N exploration agents each start from a candidate solution and iteratively apply design-wide optimization paths, to produce the final optimized design $\mathcal{D}^*$.

ods [5]–[7] build surrogate models over pragma configurations; Integer Linear Programming (ILP) and non-linear programming formulations [8], [9] compose optimized configurations from pre-enumerated candidates. These approaches navigate large configuration spaces efficiently, but cannot restructure code, rewrite algorithms, or discover optimizations outside the predefined parameter space. Recent LLM-based methods [10]–[12] are a promising step, but most still operate

within structured pragma selection rather than open-ended program transformation.

This work explores *how far general-purpose coding agents—with no HLS-specific training—can go in optimizing hardware designs, given only source code, synthesis tool access, and freedom to modify both code and pragmas*. The motivation is straightforward: recent agentic coding systems [13]–[16] have shown that LLM-based agents can iteratively refine code using tool feedback across a range of software engineering tasks, and similar agent-driven optimization efforts are emerging in adjacent domains [17]. The key question is whether this capability transfers to the hardware domain, where optimization requires reasoning about latency, area, memory bandwidth, and non-obvious pragma interactions. Unlike parameter-sweep methods, agents search over *programs*: in principle, they can restructure loops, replace computations with closed-form expressions, or reorganize memory access. In practice, as our results show, agents most often discover effective pragma strategies, but occasionally apply code-level transformations that go beyond what predefined parameter spaces can enumerate.

Our framework uses an *agent factory* that generates multiple autonomous optimization agents exploring different trajectories through the design space. The framework operates in two stages: (1) sub-kernels are independently optimized and combined via ILP under area constraints; (2) N global agents explore full-design transformations over the top-ranked ILP solutions. By varying N , we study whether agent scaling, allocating more inference-time compute, improves solution quality. We evaluate the pipeline on 12 kernels: 6 from prior HLS benchmarks [12] and 6 from Rodinia-HLS [18], using Claude Code (Opus 4.5/4.6) and AMD Vitis HLS.

We frame this as an **empirical study**: our benchmark set is small, several kernels are well-studied, and results are specific to a single model family and toolchain. The contribution is a systematic characterization of what general-purpose agents can and cannot do in hardware optimization, with the goal of establishing a baseline that the community can build upon. Our findings:

1. Agent scaling improves exploration. Increasing the number of agents from 1 to 10 yields a mean $8.27\times$ speedup over baseline, with the largest gains on complex workloads: *streamcluster* exceeds $20\times$, and *kmeans* reaches $\sim 10\times$. Simpler or pipeline-dominated kernels show limited or saturating improvements.

2. Agents go beyond isolated pragma selection. The best final design does not always originate from the top-ranked ILP variant, indicating that global optimization across function boundaries can uncover improvements not reachable by sub-kernel search alone. On *lavamd*, agents achieve $\sim 8\times$ speedup at ~ 40 – 60 K area, improving area–latency trade-offs compared to reference implementations.

3. Agents recover known hardware optimization patterns without training. Across kernels, agents consistently apply `ARRAY_PARTITION` to resolve memory bottlenecks and learn that `PIPELINE` is ineffective unless loop-carried

dependencies are first addressed, patterns that align with established HLS expertise.

4. Limitations are significant and inform future work. This is a preliminary study with inherent scope constraints. The benchmark set of twelve kernels does not capture the full complexity of real-world HLS workloads. All experiments use a single model family (Claude Opus 4.5/4.6), a single synthesis tool (Vitis HLS), and a single target architecture (FPGA). Baselines, while systematic, are bounded exhaustive searches over restricted directive sets rather than comparisons against state-of-the-art DSE frameworks such as AutoDSE. Gains are also uneven: simpler kernels saturate early, and under tight area budgets additional agents can increase area without proportional latency improvement. We view this study as a starting point intended to bootstrap a line of investigation that can be extended, through broader benchmarks, stronger baselines, additional models, and diverse target architectures, via community contribution.

II. RELATED WORK

a) Automated Design Space Exploration for HLS: Optimizing HLS designs requires tuning directives such as pipelining, unrolling, and memory partitioning, motivating extensive work on automated DSE. Early approaches relied on heuristic and analytical strategies [19], [20], followed by iterative synthesis-driven frameworks [1], lattice-based traversal [21], multi-level parallelism modeling [22], and multi-strategy autotuning [2], [23], [24]. Learning-based methods accelerate DSE through surrogate models [1], [3], [25], [26], GNN-based program representations [27]–[29], and cross-modality learning [30]. These approaches largely treat optimization as search over a predefined parameter space, limiting their ability to perform open-ended program transformations or capture global interactions across sub-kernels.

b) LLM-based Optimization for HLS: Recent work applies LLMs to HLS optimization along three directions. First, directive generation systems such as HLSPIlot, LIFT, LLM-DSE, and iDSE use LLMs to propose pragma configurations within synthesis-in-the-loop feedback [10], [11], [31], [32], but remain within predefined parameter spaces. Second, source-to-source transformation approaches iteratively convert C/C++ into synthesizable HLS code, focusing primarily on correctness and synthesizability [33], [34]. Third, agentic pipelines integrate profiling, transformation, and DSE [11], [35], but typically follow a single optimization trajectory or fixed coordination strategy.

c) Agentic LLM Systems and Scaling: LLM-based agents enable multi-step reasoning, tool use, and iterative code refinement across software engineering tasks [13]–[16]. Our work builds on this paradigm by treating the number of agents as inference-time compute, analogous to test-time scaling [36]. While prior agentic systems typically rely on a small number of coordinated agents, we study *agent scaling* as a first-class design dimension for HLS optimization.

III. METHOD

Given an input program written in a high-level language (e.g., C/C++) and a target implementation such as an FPGA platform or ASIC with resource limits, the goal of HLS design space exploration is to find an implementation that minimizes execution latency (in clock cycles) while satisfying area constraints, flip-flops, RAMs, and DSP blocks. The design space consists of all feasible combinations of HLS pragma assignments (e.g., PIPELINE, UNROLL, ARRAY_PARTITION) and code-level transformations (e.g., loop restructuring, memory reorganization) applied to the input program, where each configuration yields a distinct hardware implementation with different latency and resource utilization.

This space is difficult to explore for three reasons:

- 1) First, the number of feasible configurations grows combinatorially with the number of loops, arrays, and functions, and each configuration requires an HLS synthesis run that may take minutes, making exhaustive search impractical.
- 2) Second, optimization decisions interact globally: aggressively optimizing one function may exhaust the area budget and prevent improvements elsewhere, so exploration must balance decisions across the full design.
- 3) Third, the effect of pragmas is non-linear and sometimes counterintuitive – for example, in the Needleman–Wunsch kernel, fully unrolling the `reverse_string` loop increased latency from 26 to 71 cycles due to memory port contention, illustrating that more aggressive optimization does not always improve performance.

These challenges motivate a two-stage pipeline that decomposes the problem into independent sub-kernel optimization followed by global, design-wide exploration.

A. Notation

Let the input design $\mathcal{D}$ consist of a top-level function that invokes a set of sub-functions $\mathcal{F} = \{f_1, f_2, \dots, f_K\}$. We extract a *function call graph* $G = (\mathcal{F}, \mathcal{E})$, where a directed edge $(f_i, f_j) \in \mathcal{E}$ indicates that f_i invokes f_j . From G we compute the critical path and classify inter-function dependencies as sequential or parallel.

For each sub-function f_k , we define a *variant set* $\mathcal{V}_k = \{v_k^0, v_k^1, \dots, v_k^M\}$, where each variant v_k^m is a distinct optimization configuration (pragma assignment and/or code transformation). Each variant is characterized by its latency L_k^m and area A_k^m , obtained through HLS synthesis after passing a functional correctness check. A binary selection variable $x_k^m \in \{0, 1\}$ indicates whether variant m of sub-function k is selected.

Let $\mathcal{A}_{\text{budget}}$ denote the global area constraint imposed by the target FPGA platform, and let L_{baseline} denote the latency of the unoptimized design.

B. Stage 1: Sub-Kernel Optimization and ILP Selection

Stage 1 proceeds in three phases: variant generation, evaluation, and ILP-based selection.

TABLE I
VARIANT EXPLORATION STRATEGY PER SUB-FUNCTION.

Variant	Strategy
v_k^0	Baseline: synthesize unmodified code
v_k^1	Conservative: minimal pragmas, low area
$v_k^{2,3}$	Pipeline: PIPELINE II $\in \{1, 2, 4\}$
$v_k^{4,5}$	Aggressive: pipeline + partial/full UNROLL
v_k^6	Alternate: ARRAY_PARTITION, INLINE, closed-form rewrites

1) *Phase 1a — Variant Generation*: A coordinator agent analyzes the call graph G and spawns one *optimizer agent* per sub-function $f_k \in \mathcal{F}$. Each optimizer explores $M = 7$ variants following a structured strategy (Table I).

2) *Phase 1b — Evaluation*: Each variant v_k^m is evaluated through a pipeline of (i) functional correctness testing against the original design, (ii) HLS synthesis on the target platform, and (iii) extraction of latency L_k^m and area A_k^m . Variants that fail correctness are discarded.

3) *Phase 1c — ILP Formulation*: Once all optimizer agents complete, the coordinator aggregates the surviving variants for each sub-kernel. At this stage, each function f_k has multiple candidate implementations with latency L_k^m and area A_k^m obtained from HLS synthesis.

Selecting the best variant for each function independently is insufficient, because the latency of the overall design depends on how these functions compose in the program execution structure. In HLS designs, functions may execute sequentially, overlap through pipelining, or appear within loop iterations. As a result, the global latency cannot be expressed as a simple sum of individual function latencies.

To determine how candidate variants interact in the full design, the coordinator analyzes the program call graph G and derives a latency composition model that reflects the execution structure of the program. For sub-functions on a sequential path, latencies accumulate; for sub-functions that execute in parallel regions, the dominant latency determines the stage duration. The resulting latency model is expressed as

$$L_{\text{total}}(\mathbf{x}) = h(\{L_k^m \cdot x_k^m\}_{k,m}), \quad (1)$$

where $h(\cdot)$ encodes the latency composition derived from the call graph (e.g., sums along sequential chains and maxima over parallel branches, with appropriate loop multipliers). The binary decision variable x_k^m indicates whether variant m of sub-function f_k is selected.

Using this derived latency model, the coordinator constructs an Integer Linear Programming (ILP) formulation that selects one variant per sub-kernel while respecting the global area constraint:

$$\begin{aligned} \min_{\mathbf{x}} \quad & L_{\text{total}}(\mathbf{x}) \\ \text{s.t.} \quad & \sum_m x_k^m = 1 \quad \forall f_k \in \mathcal{F}, \\ & \sum_{k,m} A_k^m x_k^m \leq \mathcal{A}_{\text{budget}}, \quad x_k^m \in \{0, 1\}. \end{aligned} \quad (2)$$

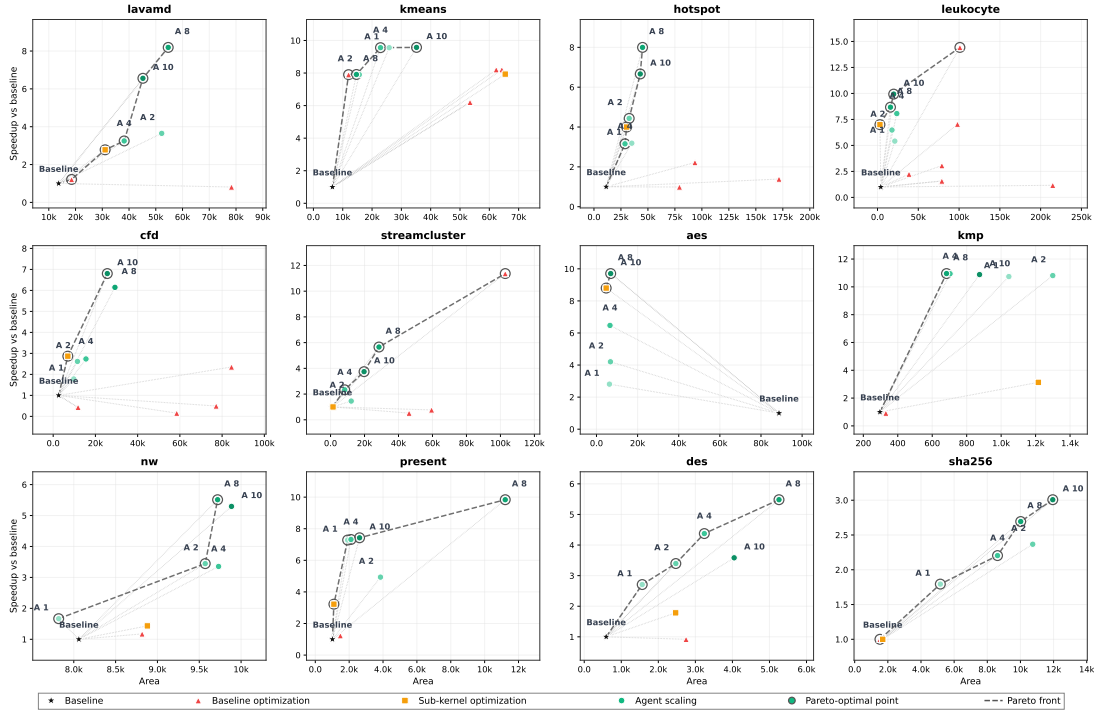


Fig. 2. Pareto front results for all twelve benchmarks under agent scaling ($N \in \{1, 2, 4, 8, 10\}$). Each subplot shows speedup over baseline (y-axis) versus area (x-axis). Increasing the number of agents extends the Pareto front toward lower latency and more favorable area-latency trade-offs across most benchmarks, with the strongest gains on harder problems such as streamcluster, leukocyte, and NW.

Because multiple variant combinations can satisfy the area constraint while achieving similar latency, the solver is used to enumerate the top- N feasible solutions ranked by L_{total} , denoted $\mathcal{S} = \{s_1, s_2, \dots, s_N\}$. Each solution corresponds to a distinct global configuration of sub-kernel implementations and serves as a starting point for the next phase of full-design optimization.

C. Stage 2: Global Optimization via Agent Scaling

Stage 2 takes the candidate set $\mathcal{S}$ from the ILP solver and attempts to improve each candidate through holistic, design-wide optimization.

A coordinator agent spawns N exploration agents, one per candidate $s_i \in \mathcal{S}$. Each agent receives the full design instantiated with the sub-kernel variants specified by s_i and explores optimization paths that operate across function boundaries. These paths are categorized as:

1. **Pragma composition:** new combinations of HLS pragmas applied jointly across multiple functions, not explored during Stage 1.
2. **Code restructuring:** loop reordering, loop fusion, or function inlining at the global level.
3. **Memory optimization:** cross-function array partitioning and memory access restructuring.
4. **Compute optimization:** algebraic simplifications or closed-form transformations spanning multiple sub-kernels.

Each agent iteratively generates modified designs, verifies correctness, synthesizes via HLS, and records (latency, area)

pairs. Let $\mathcal{R}_i$ denote the set of all valid designs explored by agent i . The final output of the pipeline is:

$$\mathcal{D}^* = \arg \min_{d \in \bigcup_{i=1}^N \mathcal{R}_i} L(d) \quad \text{s.t.} \quad A(d) \leq \mathcal{A}_{\text{budget}}. \quad (3)$$

D. Algorithm Summary

Algorithm 1 in the appendix summarizes the complete two-stage pipeline. By parameterizing Stage 2 with N , we study how increasing the number of parallel exploration agents affects solution quality. Larger N expands the explored region of the design space, increasing the probability of discovering lower-latency implementations. We evaluate $N \in \{1, 2, 4, 8, 10\}$ in Section IV.

IV. RESULTS

A. Experimental Setup

We evaluate the two-stage pipeline on twelve HLS kernels: six HLS-Eval benchmarks from prior work [12] (AES, DES, KMP, NW, PRESENT, SHA256) and six from Rodinia-HLS [18] (lavamd, kmeans, hotspot, leukocyte (lc_dilate), cfd (cfd_step_factor), streamcluster). Results are averaged over 5 runs; when HLS reports latency as a range, we use the midpoint. All experiments use Claude Code (Opus 4.5/4.6) with AMD Vitis HLS.

a) *Baselines:* For HLS-Eval kernels, we use a bounded exhaustive baseline that enumerates per-loop pragma configurations for each sub-kernel. Each loop selects one of five options: no directive, PIPELINE (II $\in \{1, 2\}$), or UNROLL

TABLE II
MEAN SPEEDUP OVER BASELINE AS AGENTS SCALE

	1→2	2→4	4→8	8→10
Speedup	4.31× (+5.9%)	4.92× (+14.2%)	7.07× (+43.7%)	6.53× (-7.6%)

(factor $\in 2, 4$), capturing common loop optimizations with a manageable branching factor. The search space grows as 5^n for n loops, so we cap variants per function to keep synthesis tractable. Each variant is synthesized to obtain latency and area, and an ILP selects one variant per sub-kernel to minimize latency under a global area constraint. We design the baseline to operate at the pragma level, isolating directive search and enabling a controlled evaluation of the additional benefits from higher-level transformations (e.g., code restructuring, memory layout, array partitioning) introduced by the LLM agent. This provides a strong pragma-only reference, though not globally optimal due to the bounded search.

For Rodinia-HLS kernels, we compare against reference optimized implementations (e.g., tiling, pipelining, double-buffering) and report the best feasible design.

All designs must satisfy area and timing constraints. We use a fixed clock of 10ns (35ns for *streamcluster*); infeasible baseline configurations are adjusted by reducing pragma factors, and designs violating area or timing are discarded.

B. Results on FPGA using Vitis HLS

1) *Sub-Kernel Optimization*: Across the twelve benchmarks, Stage 1 improves latency over the baseline while remaining within the area budget; SHA256 is the exception, where performance stays comparable to baseline (Fig. 2).

Two recurring patterns emerge without any HLS-specific training. First, agents consistently identify `ARRAY_PARTITION` as the highest-impact directive, yielding the largest gains in AES, DES, and PRESENT by resolving memory bottlenecks. Second, agents learn that `PIPELINE` applied in isolation is often ineffective—or even harmful—unless memory bandwidth and loop-carried dependencies are addressed first.

2) *ILP Variant Selection*: Following sub-kernel optimization, a coordinator agent composes the surviving variants into a full design. It analyzes the function call graph, derives a latency composition model that accounts for sequential and parallel execution paths, and formulates the ILP objective (Eq. 2) to minimize global latency under an area constraint.

Table III validates the agent’s inferred latency structure on two synthetic benchmarks (SYN5, SYN6) from [12] and two real kernels (NW, AES). The agent correctly captures parallel compositions—minimizing the maximum latency for concurrent modules (SYN5)—and mixed parallel-sequential structures (SYN6).

Because this formulation estimates global latency from isolated sub-kernel synthesis, it cannot capture cross-function effects such as inter-kernel memory reuse or global pipeline

TABLE III
THE AGENT ANALYZES DATA FLOW GRAPHS AND AUTOMATICALLY FORMULATES ILP CONSTRAINTS TO MINIMIZE LATENCY UNDER AN AREA BUDGET. DATAFLOW GRAPH BORROWED FROM [12].

DFG	Agent-formulated ILP & constraints
	$\begin{aligned} \min & L_{iter} \cdot \text{Loop_count} \\ \text{s.t. } & L_{iter} \geq L_F + L_O, \quad L_{iter} \geq 2L_F, \quad L_{iter} \geq L_E + L_F \\ & \sum_v x_v = 1, \quad \sum_v A_v x_v \leq A_{budget}, \quad x_v \in \{0, 1\} \end{aligned}$ One variant per function; A_v is variant area.
	$\begin{aligned} \min & 5(M_{FO} + 2L_E) \\ \text{s.t. } & M_{FO} \geq L_F, \quad M_{FO} \geq L_O \\ & \sum_v x_v = 1, \quad \sum_v A_v x_v \leq A_{budget}, \quad x_v \in \{0, 1\} \end{aligned}$ L_F, L_O denote latencies of F and O.
	$\begin{aligned} \min & (L_{FM} + L_{TB} + L_{RS}) \\ & \sum_v x_v = 1, \quad \sum_v A_v x_v \leq A_{budget}, \quad x_v \in \{0, 1\} \end{aligned}$ L_{FM}, L_{TB}, L_{RS} denote module latencies.
	$\begin{aligned} \min & 11L_{ARK} + 10L_{SB} + 10L_{SR} + 9L_{MC} + L_{KE} + L_{INIT} \\ & \sum_v x_v = 1, \quad \sum_v A_v x_v \leq A_{budget}, \quad x_v \in \{0, 1\} \end{aligned}$ L_* denotes module latency.

scheduling. Exhaustive enumeration would resolve this gap but is computationally prohibitive. The ILP stage therefore serves as an efficient filter that narrows the search space, producing a ranked set of N starting points for Stage 2.

3) *Global Optimization*: Figure 2 shows Pareto fronts across all twelve benchmarks, and Table II reports the mean best speedup over 5 runs as agents scale from $N=1$ to $N=10$. We organize the findings below by the character of the scaling behavior.

a) *Strong scaling on complex workloads*: The largest gains appear on benchmarks with rich optimization landscapes, though improvements are more moderate than initially observed. *Streamcluster* shows consistent benefits from scaling, with speedup reaching $\sim 5\text{--}6\times$ within the evaluated range of up to 10 agents, indicating continued gains without clear evidence of saturation. *Cfd*, *hotspot*, and *kmeans* follow a similar pattern, reaching $7\text{--}10\times$ speedup with progressively better area-latency trade-offs. Among the larger Rodinia kernels, *lavamd* reaches $\sim 8\times$ speedup at moderate area.

b) *Intermediate and non-monotonic behavior*: For PRESENT, NW, and DES, scaling improves latency overall but the progression is not strictly monotonic: in several cases (e.g., PRESENT, DES), $N=8$ yields Pareto-optimal designs not dominated by $N=10$, reflecting stochastic exploration effects. *Leukocyte* presents a distinct pattern: its top speedup ($\sim 14\text{--}15\times$) originates from a baseline optimization that agents do

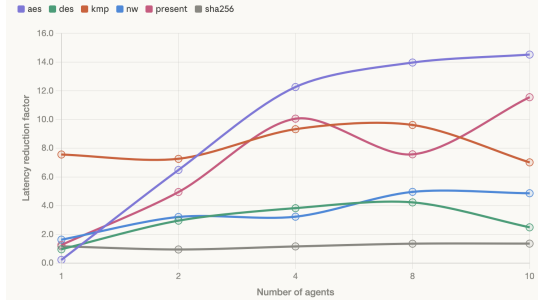


Fig. 3. Latency improvement factor over baseline versus number of expert agents across six benchmarks. Improvements range from 1.4 $\times$ to 14.5 $\times$ and generally increase with agent count, typically plateauing after four agents.

not replicate, though they do improve designs in the ~ 7 – $10\times$ range.

c) Saturation under simplicity or tight resource budgets:

Simpler or resource-constrained kernels exhibit diminishing returns. *KMP* saturates early (~ 10 – $12\times$), and *AES* shows similar performance for $N=8$ and $N=10$. *NW*, which operates near its area limit ($\sim 10K$), illustrates a related effect: $N=8$ slightly outperforms $N=10$, suggesting that under tight area budgets additional agents may explore configurations that trade area for diminishing latency gains.

d) Global optimization beyond sub-kernel decomposition:

Across benchmarks, the best final designs do not always originate from the top-ranked ILP variant. This confirms that Stage 2 optimization—through pragma recombination and code-level transformations applied across function boundaries—can exploit cross-function interactions that the ILP model, which estimates global latency from isolated sub-kernel synthesis, cannot capture.

C. Generalization to ASICs

The approach presented in this paper uses HLS tools with FPGA as the target device to evaluate and guide the agent-based optimization flow. While the approach itself is tool-agnostic, and can use ASIC-targeted HLS and logic synthesis tools to evaluate and guide the flow, we show that both latency improvements and area correlate well to an ASIC-mapped design as well.

To measure how well area reported by HLS tool correlates with ASIC-mapped area, we compute correlation between HLS-reported synthesis area and logic area obtained from the open-source logic synthesis tool ABC, across the six HLS-Eval benchmarks. The strength of the correlation varies: *SHA256* ($r=0.992$), *KMP* ($r=0.964$), and *NW* ($r=0.859$) show strong near-linear relationships, indicating that HLS area estimates are generally reliable proxies for silicon cost on these kernels. *AES* ($r=0.757$) shows moderate-to-strong correlation, while *DES* ($r=0.603$) is moderate. *PRESENT* ($r=0.277$) exhibits weak correlation, suggesting that HLS area estimates may be less reliable for this design. This variation is because HLS tool area estimates tend to be inaccurate for designs with more memory instances.

Figure 3 shows the factor of improvement in latency (compared to the baseline design) on the y-axis and number of

agents on the x-axis. Latency is computed as the product of number of cycles obtained from the HLS timing report and an estimate for clock period obtained from ABC synthesis report. The latency value for different number of agents is then normalized by the latency of the baseline design (without agentic optimization) to get the latency reduction factor on the y-axis. Across the six benchmarks, latency can be improved by a factor ranging from 1.4 $\times$ to 14.5 $\times$. For all benchmarks, latency improvements increase with increasing number of expert agents, typically plateauing after four expert agents.

D. Discussion and Future Work

This work presents an initial exploration of agent scaling for HLS design space exploration. Our results demonstrate that *Agent Factories*, when applied to High Level Synthesis domain yield several consistent trends: increasing the number of agents improve exploration and extend the Pareto frontier on several kernels. This suggests that agent scaling, analogous to inference-time scaling in LLMs can serve as a powerful mechanism for navigating large hardware designs spaces without explicit training or handcrafted heuristics. While these results are encouraging, the current study can be significantly expanded in scope. Our evaluation spans twelve benchmarks and focuses on Vitis HLS with logic-level validation through ABC, without yet incorporating broader benchmark suites (e.g., HLSyn) or comparisons to advanced automated DSE frameworks such as AutoDSE.

Several directions follow naturally from this observation: incorporating learning (e.g., fine-tuning or reinforcement learning) to improve generalization and efficiency; expanding evaluation across broader benchmarks and toolchains, including downstream synthesis; and integrating advanced agentic strategies (e.g., memory, replay, coordination) to improve search quality. In addition, we are working towards several baselines comparisons with traditional optimization methods as well for HLS design space exploration to determine the differentiation and tradeoffs involved with *Agent Factories*.

This research demonstrates a powerful result that taken together, the direction point toward a broader vision in which agent-based systems, combined with learning and deeper integration into hardware toolchains, act as scalable optimizers for complex design spaces, reducing the need for manual expertise and enabling more automated hardware design.

APPENDIX A AGENT SCALING TOKEN USAGE

Token usage is reported as the total number of model tokens consumed by the agents during execution, with Claude Opus 4.5 and Claude Opus 4.6 used as the underlying models. Across all valid runs, the method consumed a median of 5.82M tokens and a mean of 7.67M tokens per run, indicating a right-skewed distribution with a small number of substantially more expensive runs. The 25th and 90th percentiles were 3.09M and 13.39M tokens, respectively, while the observed range spanned from 1.14M to 45.33M tokens. These results

indicate that token consumption is typically in the low-to-mid millions per run, with occasional substantially higher-cost outliers. The overall token summary is shown in Fig. 4.

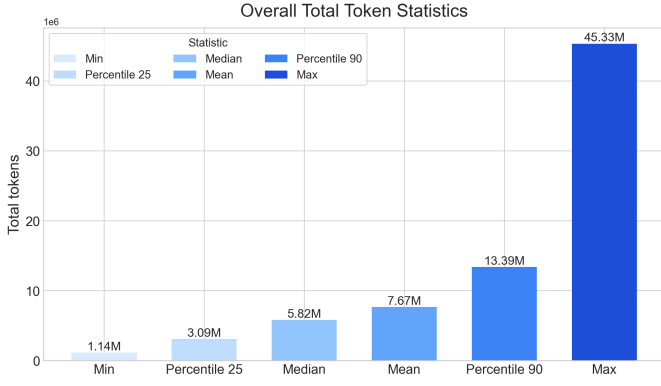


Fig. 4. Average Inference cost of agent scaling. Each session is combination of Opus 4.5/4.6.

APPENDIX B ALGORITHM

Algorithm 1 summarizes the complete two-stage pipeline. By parameterizing Stage 2 with N , we study how increasing the number of parallel exploration agents affects solution quality. Larger N expands the explored region of the design space, increasing the probability of discovering lower-latency implementations. We evaluate $N \in \{1, 2, 4, 8, 10\}$ in Section IV.

REFERENCES

- [1] A. Sohrabizadeh, C. H. Yu, M. Gao, and J. Cong, “Autodse: Enabling software programmers to design efficient fpga accelerators,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 27, no. 4, Feb. 2022. [Online]. Available: <https://doi.org/10.1145/3494534>
- [2] J. Ansel, S. Kamil, K. Veeramachaneni, J. Ragan-Kelley, J. Bosboom, U.-M. O’Reilly, and S. Amarasinghe, “Opentuner: an extensible framework for program autotuning,” in *Proceedings of the 23rd International Conference on Parallel Architectures and Compilation*, ser. PACT ’14. New York, NY, USA: Association for Computing Machinery, 2014, p. 303–316. [Online]. Available: <https://doi.org/10.1145/2628071.2628092>
- [3] J. Zhao, L. Feng, S. Sinha, W. Zhang, Y. Liang, and B. He, “Comba: a comprehensive model-based analysis framework for high level synthesis of real applications,” in *Proceedings of the 36th International Conference on Computer-Aided Design*, ser. ICCAD ’17. IEEE Press, 2017, p. 430–437.
- [4] J. Cong, J. Lau, G. Liu, S. Neuendorffer, P. Pan, K. Vissers, and Z. Zhang, “Fpga hls today: Successes, challenges, and opportunities,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 15, no. 4, Aug. 2022. [Online]. Available: <https://doi.org/10.1145/3530775>
- [5] M. R. Ahmed, T. Koike-Akino, K. Parsons, and Y. Wang, “Autohls: Learning to accelerate design space exploration for hls designs,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.10686>
- [6] H. Kuang, X. Cao, J. Li, and L. Wang, “Hgbo-dse: Hierarchical gnn and bayesian optimization based hls design space exploration,” in *2023 International Conference on Field-Programmable Technology (ICFPT)*, 2023, pp. 106–114.
- [7] H. Kuang and L. Wang, “Compass: A collaborative hls design space exploration framework via graph representation learning and ensemble bayesian optimization,” *2024 International Conference on Field Programmable Technology (ICFPT)*, pp. 1–9, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:280697200>

Algorithm 1: Two-Stage Multi-Agent Design Space Exploration for HLS

Input: Design $\mathcal{D}$, area budget $\mathcal{A}_{\max}$, agents N

Output: Optimized design $\mathcal{D}^*$

```

1 Coordinator extracts call graph  $G = (\mathcal{F}, \mathcal{E})$  from  $\mathcal{D}$ ;
2 forall  $f_k \in \mathcal{F}$  in parallel do
3   launch optimizer agent  $a_k$ ;
4    $\mathcal{V}_k \leftarrow a_k.\text{SEARCHANDEVALUATE}(f_k)$ ;
5 end
6 ILP agent builds a global model from  $\{\mathcal{V}_k\}$  and  $G$ ;
7  $\mathcal{S} \leftarrow$  top- $N$  feasible ILP solutions;
8 forall  $s_i \in \mathcal{S}$  in parallel do
9   launch exploration agent  $g_i$ ;
10   $\mathcal{R}_i \leftarrow g_i.\text{TARGETEDREFINEMENT}(\mathcal{D}, s_i, \mathcal{A}_{\max})$ ;
11 end
12  $\mathcal{D}^* \leftarrow \arg \min_{d \in \cup_i \mathcal{R}_i} L(d)$  s.t.  $A(d) \leq \mathcal{A}_{\max}$ ;
13 return  $\mathcal{D}^*$ ;
14 Function SEARCHANDEVALUATE( $f_k$ ):
15   generate candidate variants  $\{v_k^m\}_{m=0}^M$ ;
16    $\mathcal{V}_k \leftarrow \{(v_k^m, L^m, A^m) \mid \text{CORRECTNESS}(v_k^m)\}$ ;
17   return verified variants in  $\mathcal{V}_k$  with HLS metrics;
18 Function TARGETEDREFINEMENT( $\mathcal{D}, s, \mathcal{A}_{\max}$ ):
19   instantiate  $\mathcal{D}_s \leftarrow \text{INSTANTIATE}(\mathcal{D}, s)$ ;
20   generate refinement attempts  $\{p_j\}$ ;
21    $\mathcal{R} \leftarrow \{(\mathcal{D}_j, L^j, A^j) \mid \mathcal{D}_j =$ 
        APPLY( $p_j, \mathcal{D}_s$ ), CORRECTNESS( $\mathcal{D}_j$ ),  $A^j \leq$ 
         $\mathcal{A}_{\max}\}$ ;
22   return  $\mathcal{R}$  if non-empty, else  $\{(\mathcal{D}_s, L_s, A_s)\}$ ;

```

- [8] V. G. Castellana, A. Tumeo, and F. Ferrandi, “High-level synthesis of parallel specifications coupling static and dynamic controllers,” in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2021, pp. 192–202.
- [9] S. Pouget, L.-N. Pouchet, and J. Cong, “Automatic hardware pragma insertion in high-level synthesis: A non-linear programming approach,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 30, no. 2, Feb. 2025. [Online]. Available: <https://doi.org/10.1145/3711847>
- [10] N. Prakriya, Z. Ding, Y. Sun, and J. Cong, “Lift: Llm-based pragma insertion for hls via gnn supervised fine-tuning,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.21187>
- [11] C. Xiong, C. Liu, H. Li, and X. Li, “Hlspilot: Llm-based high-level synthesis,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.06810>
- [12] L. Collini, A. Hennessee, R. Karri, and S. Garg, “Can reasoning models reason about hardware? an agentic hls perspective,” in *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*, 06 2025, pp. 188–194.
- [13] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.15793>
- [14] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “Metagpt: Meta programming for a multi-agent collaborative framework,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.00352>
- [15] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, “Chatdev: Communicative agents for software development,” 2024. [Online]. Available: <https://arxiv.org/abs/2307.07924>
- [16] S. Wong, Z. Qi, Z. Wang, N. Hu, S. Lin, J. Ge, E. Gao, W. Chen, Y. Du, M. Yu, and Y. Zhang, “Confucius code agent: Scalable agent scaffolding for real-world codebases,” 2026. [Online]. Available:

<https://arxiv.org/abs/2512.10398>

- [17] A. Karpathy, “Autoresearch: Autonomous machine learning research with ai agents,” <https://github.com/karpathy/autoresearch>, 2026.
- [18] J. Cong, Z. Fang, M. Lo, H. Wang, J. Xu, and S. Zhang, “Understanding performance differences of fpgas and gpus: (abstract only),” in *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA ’18. New York, NY, USA: Association for Computing Machinery, 2018, p. 288. [Online]. Available: <https://doi.org/10.1145/3174243.3174970>
- [19] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, “High-level synthesis for fpgas: From prototyping to deployment,” *IEEE Transactions on Computer-Aided Design*, 2011.
- [20] J. Cong, P. Wang, and Y. Zhang, “Automatic design space exploration for high-level synthesis,” in *Design Automation Conference (DAC)*, 2012.
- [21] L. Ferretti, G. Ansaloni, and L. Pozzi, “Lattice-traversing design space exploration for high level synthesis,” in *2018 IEEE 36th International Conference on Computer Design (ICCD)*, 2018, pp. 210–217.
- [22] G. Zhong, A. Prakash, S. Wang, Y. Liang, T. Mitra, and S. Niar, “Design space exploration of fpga-based accelerators with multi-level parallelism,” in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, 2017, pp. 1141–1146.
- [23] C. Xu, G. Liu, R. Zhao, S. Yang, G. Luo, and Z. Zhang, “A parallel bandit-based approach for autotuning fpga compilation,” in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA ’17. New York, NY, USA: Association for Computing Machinery, 2017, p. 157–166. [Online]. Available: <https://doi.org/10.1145/3020078.3021747>
- [24] Q. Gautier, A. Althoff, C. L. Crutchfield, and R. Kastner, “Sherlock: A multi-objective design space exploration framework,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 27, no. 4, Mar. 2022. [Online]. Available: <https://doi.org/10.1145/3511472>
- [25] Y. Bai, A. Sohrabizadeh, Z. Qin, Z. Hu, Y. Sun, and J. Cong, “Towards a comprehensive benchmark for high-level synthesis targeted to fpgas,” in *Advances in Neural Information Processing Systems*, 2023.
- [26] Y. Bai, A. Sohrabizadeh, Y. Sun, and J. Cong, “Improving gnn-based accelerator design automation with meta learning,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1347–1350. [Online]. Available: <https://doi.org/10.1145/3489517.3530629>
- [27] A. Sohrabizadeh, Y. Bai, Y. Sun, and J. Cong, “Automated accelerator optimization aided by graph neural networks,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 55–60. [Online]. Available: <https://doi.org/10.1145/3489517.3530409>
- [28] —, “Robust GNN-based representation learning for HLS,” in *Proceedings of the 42nd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023.
- [29] N. Wu, Y. Xie, and C. Hao, “Ironman: Gnn-assisted design space exploration in high-level synthesis via reinforcement learning,” in *Proceedings of the 2021 Great Lakes Symposium on VLSI*, ser. GLSVLSI ’21. ACM, Jun. 2021, p. 39–44. [Online]. Available: <http://dx.doi.org/10.1145/3453688.3461495>
- [30] Z. Qin, Y. Bai, A. Sohrabizadeh, Z. Ding, Z. Hu, Y. Sun, and J. Cong, “Cross-modality program representation learning for electronic design automation with high-level synthesis,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.09606>
- [31] H. Wang, X. Wu, Z. Ding, S. Zheng, C. Wang, N. Prakriya, T. Nowatzki, Y. Sun, and J. Cong, “Llm-dse: Searching accelerator parameters with llm agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.12188>
- [32] R. Li, J. Xiong, and X. Wang, “idse: Navigating design space exploration in high-level synthesis using llms,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.22086>
- [33] L. Collini, S. Garg, and R. Karri, “C2hlsc: Leveraging large language models to bridge the software-to-hardware design gap,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 6, p. 1–24, Oct. 2025. [Online]. Available: <http://dx.doi.org/10.1145/3734524>
- [34] K. Xu, G. L. Zhang, X. Yin, C. Zhuo, U. Schlichtmann, and B. Li, “Automated c/c++ program repair for high-level synthesis via large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.03889>
- [35] A. E. Oztas and M. Jelodari, “Agentic-hls: An agentic reasoning based high-level synthesis system using large language models (ai for eda workshop 2024),” 2024. [Online]. Available: <https://arxiv.org/abs/2412.01604>
- [36] I. Puri, S. Sudalairaj, G. Xu, K. Xu, and A. Srivastava, “Rollout roulette: A probabilistic inference approach to inference-time scaling of llms using particle-based monte carlo methods,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.